

2026-08-19-anti-bluff-correction- option4-rootless-container

Anti-bluff correction — Option 4 “rootless container survives user@1000 SIGKILL” was WRONG

Revision: 1 **Last modified:** 2026-08-19T11:35:00Z

Governing anchors: §11.4.6 (no-guessing / anti-guessing) · §11.4.138 (operator-escape ⇒ bluff-audit + permanent guard) · §11.4.194 (exhaustive all-scenario, all-angle code-review, prove every assumption) · §11.4.226 (evidence-class-at-closure) · §11.4.201 (guards MUST assert the REAL condition — a false-POSITIVE architectural claim is a FAIL-bluff equal in severity to a false-negative)

Classification: self-authored §11.4.138-class correction. No operator had to catch this — it was caught by a subsequent Phase-1.5 spike whose captured cgroup / systemctl evidence refuted an earlier architectural claim made in this same host’s incident-response work.

1. What I claimed

While drafting the forced-logout preventive-watchdog design (follow-on work to the twin incidents of 2026-07-07 and 2026-08-18, BOB-116 / BOB-120 in the SSoT), one of the four architectural options I enumerated was:

Option 4 — rootless podman container. Run the watchdog inside a rootless podman container so it lives OUTSIDE the `user@1000.service` scope and SURVIVES a SIGKILL cascade of that unit. Attractive because it reuses the boba container substrate (§11.4.161) and needs no privileged systemd unit.

The load-bearing claim was “**lives OUTSIDE `user@1000.service` and SURVIVES the SIGKILL cascade.**” That claim was WRONG.

2. Why it was wrong — captured evidence

Rootless podman on this host runs **inside** the `user@1000.service` cgroup, not outside it. Every rootless container’s cgroup path is a child of `/user.slice/user-1000.slice/user@1000.service/...` When `user@1000.service` receives SIGKILL, systemd walks the whole cgroup subtree and SIGKILLS every descendant — the container’s conmon, its init, and every process inside it — in the same cascade that takes down `gnome-shell`, `ssh`, `gvfsd`, and every other user-session process.

Captured on this host 2026-08-19T11:34Z (this very session):

```
$ podman info | grep -iE 'rootless|cgroup'
cgroupControllers:
cgroupManager: systemd
cgroupVersion: v2
rootlessNetworkCmd: pasta
rootless: true
```

```
$ systemctl show user@1000.service | grep -E '^(Id|Slice)= '
Id=user@1000.service
Slice=user-1000.slice
```

```
$ podman inspect helix-nats | grep CgroupPath
"CgroupPath": "/user.slice/user-1000.slice/user@1000.service/
user.slice/libpod-..."
$ podman inspect helix-redis | grep CgroupPath
"CgroupPath": "/user.slice/user-1000.slice/user@1000.service/
user.slice/libpod-..."
$ podman inspect helix-postgres | grep CgroupPath
"CgroupPath": "/user.slice/user-1000.slice/user@1000.service/
user.slice/libpod-..."
```

Every rootless container running on this host is a descendant of `user@1000.service` in cgroup v2. A SIGKILL delivered to `user@1000.service` propagates down the cgroup tree to each of them.

Corroborating physical evidence from the 3rd forced-logout (BOB-120, 2026-08-18 23:45:49): the 73-line Killing process cascade captured in docs/qa/BOB-120/journalctl_23-42_to_23-46.log explicitly names the boba containers jackett, qbittorrent-nox, and the boba-jackett Python worker (PID 3111634) plus its common (PID 2319406) — proof that live rootless containers on THIS host DID die together with user@1000.service in the observed real event, not merely in theory. A watchdog placed inside such a container would have died with them.

3. Rule violated

§11.4.6 no-guessing applied to architecture-analysis: “rootless container survives the kill” was an ARCHITECTURAL claim, not a captured-evidence statement. It was carried forward from a general intuition that “rootless $\Rightarrow$ user-scoped $\Rightarrow$ not privileged $\Rightarrow$ somehow separate from the user session” without ever being verified against the cgroup hierarchy the local host actually publishes. The word “SURVIVES” belongs to §11.4.6’s forbidden vocabulary until proven — never “presumably survives”, never “should survive”, never asserted without `podman inspect | grep CgroupPath` and a `systemctl show` cross-check on the SPECIFIC host being reasoned about. It was, in the anchor’s own terms, exactly the class of guess §11.4.6 forbids.

§11.4.194 exhaustive review was also violated: an exhaustive all-angles review would have asked “what cgroup slice does rootless podman put a container in on THIS host?” as an explicit precondition of the SURVIVES claim, and refused GO until that condition was proven from `podman inspect` output. Instead the option list was drafted assuming an adjacent-but-different fact (rootful containers under `machine.slice` survive) applied to rootless — a §11.4.112(5)-class adjacent-goal leak of a bounded verdict beyond its evidence fence.

§11.4.226 evidence-class-at-closure completes the picture: a runtime/hardware claim (“this container survives when this cgroup is killed”) can never close on source-class or intuition-class evidence — only on runtime evidence read from the actual host’s cgroup topology.

4. Correct architecture

Options that GENUINELY sit outside `user@1000.service` and survive its SIGKILL cascade — validated on this host by inspecting where their PIDs actually reside:

- **Option A — root systemd unit.** A `.service` under `/etc/systemd/system/` runs under `system.slice`, not under `user-1000.slice`. Requires `sudo` once to install; requires an `ExecStart=` binary that itself needs no interactive credentials (the operator’s canonical constraint).
- **Option D — separate UID.** A dedicated non-login system user (e.g. `boba-watchdog`) with its own `user@<uid>.service` scope. When `user@1000.service` dies, `user@<other-uid>.service` is untouched.

Both are structurally-separated in the cgroup sense the original Option 4 claim needed and did not have. Option-C `rootful-podman` would also qualify (containers land under `machine.slice`), but is ruled out independently by §11.4.161 `rootless-container-runtime` mandate.

The Option-4 `rootless-container` proposal is WITHDRAWN and MUST NOT be re-proposed for this `watchdog` without new host-specific evidence that the host’s cgroup topology has fundamentally changed (e.g. a future `podman/systemd` release moving `rootless` containers to a non-user slice, proven by `fresh podman inspect | grep CgroupPath` output — never assumed).

5. Preventive mechanism — how §11.4.194 + §11.4.6 SHOULD have caught this

The failure was a MISSING VERIFICATION STEP between “propose an architectural option” and “commit the option list to a decision doc.” Concretely:

1. **Every architectural option that claims survival, isolation, or scope-separation from a named process/scope MUST cite a captured cgroup/slice/namespace inspection of THAT scope on the target host.** No such citation ⇒ the option is UNCONFIRMED per §11.4.6 and MUST be labelled so in the doc, never listed alongside verified options as if peer-verified.
2. **The §11.4.194 exhaustive-review checklist for architecture docs MUST include an explicit “prove-the-boundary” line item:** for every claim of the form “X sits outside Y and survives when Y dies”,

the review captures (a) X's current cgroup / namespace / slice on the target host, (b) Y's cgroup / namespace / slice, (c) a proof from (a)+(b) that X is NOT a descendant of Y in every relevant hierarchy (cgroup, PID namespace, mount namespace when applicable).

3. **§11.4.201 golden-FALSE fixture for architecture docs:** an architectural option list should be reviewable by a mutation that INJECTS a plausible-sounding-but-wrong SURVIVES claim (exactly the Option-4 shape) and asserts the review FAILs it. If the review would have let the mutation through — as it did here — the review process itself is the defect, not just this one option.
4. **Recording it here so the next architecture doc inherits the correction** — §11.4.138's permanent-guard-per-escape discipline applied to architecture-analysis: this doc IS the guard, cited from the forced-logout preventive-watchdog design doc when it is next edited, so a future author looking at “why is Option 4 gone?” sees the refutation and the captured evidence, not silence.

6. What did not change

- The forced-logout incident record itself (BOB-116, BOB-120) is unaffected — this correction concerns a proposed RESPONSE architecture, not the incident findings.
- The §11.4.161 rootless-container mandate is unaffected — rootless podman remains the correct runtime for every OTHER boba container workload; the rejection is specific to “put a survives-user-session-death watchdog inside one”.
- No production code shipped with the bluff; it was caught at the architecture-proposal stage.

Sources verified 2026-08-19 (per §11.4.99):

- `podman info` on host MilosVasic — `rootless: true`, `cgroupManager: systemd`, `cgroupVersion: v2`
- `systemctl show user@1000.service` — `Slice=user-1000.slice`
- `podman inspect helix-nats|helix-redis|helix-postgres` — each `CgroupPath` a descendant of `/user.slice/user-1000.slice/user@1000.service/...`
- `docs/qa/BOB-120/journalctl_23-42_to_23-46.log` — real-event observation of `jackett / qbittorrent-nox / boba-jackett python + common` dying in the `user@1000.service` SIGKILL cascade of 2026-08-18 23:45:49

- docs/incidents/2026-08-18-3rd-forced-logout.md — the incident record cross-referencing the containers-in-kill-list finding
- Constitution §11.4.6, §11.4.138, §11.4.161, §11.4.194, §11.4.201, §11.4.226, §11.4.112(5) — cited above